Notio

MSA 아키텍처 설계서

Microservices Architecture — 인프라 · 서비스 · 기술스택 · 기능 정의

Strangler Fig 패턴으로 모놀리스에서 MSA로 점진적 전환. 각 서비스는 독립 배포 · 독립 스케일 · 독립 장애 격리를 원칙으로 한다. 백엔드는 Java(Spring Boot) 기반이며 AI Service만 Python(FastAPI)으로 분리하는 하이브리드 전략을 채택한다.

항목	내용
버전	v1.0 | 2025
기반 문서	Notio 설계서 v1.0 + MSA 전환 설계
현재 단계	Phase 0 — Spring Boot 모놀리스 (MVP)
전환 목표	서비스 독립 배포 · 확장 · 기술 선택 자유도
백엔드 언어	Java 21 (메인) + Python 3.12 (AI Service)

1. MSA 전환 전략

1.1 전환 원칙

Strangler Fig Pattern: 모놀리스를 한 번에 분리하지 않고 도메인별로 순차 추출한다. API Gateway를 먼저 세우고 특정 경로만 새 마이크로서비스로 라우팅한다. 서비스 간 통신은 REST(동기) 또는 Kafka 이벤트(비동기)만 사용하며 DB는 절대 공유하지 않는다.

1.2 전환 단계 로드맵

단계	시기	분리 서비스	방식	설명
Phase 0	현재	Spring Boot 모놀리스	—	MVP 완성. 모든 도메인 단일 서비스
Phase 1	3~6개월	AI Service 분리	신규 서비스	LLM·RAG 부하 독립. Python FastAPI
Phase 2	6~9개월	Notification + Webhook 분리	도메인 추출	소스 확장 시 독립 배포
Phase 3	9~12개월	Chat + Todo 분리	도메인 추출	채팅·할일 독립 스케일
Phase 4	12개월+	Analytics + Auth + Gateway	신규 서비스	멀티유저·API 관리 완성

핵심 원칙 — 기존 코드 무수정 확장(OCP): WebhookHandler 인터페이스 구현체 추가로 새 소스 연동. LlmProvider 인터페이스로 LLM 교체. Repository 인터페이스로 스토리지 교체. 모두 기존 코드 수정 없이 가능.

2. 전체 MSA 구성

2.1 서비스 레이어

Flutter 앱은 API Gateway 단일 엔드포인트만 호출한다. Gateway가 JWT 검증 · 라우팅 · Rate Limiting · 로드밸런싱을 담당하며, 서비스 간 동기 통신은 OpenFeign(REST), 비동기 통신은 Kafka를 사용한다.

레이어	서비스	역할	언어 / 프레임워크
Client	Flutter App (iOS/Android)	UI · 상태관리 · 로컬 캐시	Flutter / Dart
Gateway	API Gateway	JWT 인증 · 라우팅 · Rate Limit · LB	Spring Cloud Gateway
Core	Notification Service	알림 수집 · 조회 · 상태 관리	Spring Boot / Java 21

레이어	서비스	역할	언어 / 프레임워크
Core	Webhook Service	Webhook 수신 · 검증 · 파싱	Spring Boot / Java 21
Core	Chat Service	채팅 세션 · 이력 · AI 연동	Spring Boot / Java 21
Core	Todo Service	할일 생성 · 관리	Spring Boot / Java 21
AI	AI Service	임베딩 · 벡터검색 · LLM 추론 · 요약	FastAPI / Python 3.12
Platform	Push Service	FCM / APNs 푸시 발송	Spring Boot / Java 21
Platform	Analytics Service	통계 집계 · LLM 주간 리포트	Spring Boot / Java 21
Platform	Auth Service	사용자 인증 · JWT 발급	Spring Boot / Java 21
Infra	Kafka · PostgreSQL · Redis · Ollama	이벤트 버스 · DB · 캐시 · LLM 런타임	Docker / Managed

2.2 서비스 간 통신 패턴

통신 유형	패턴	서비스 쌍	이유
동기 REST	OpenFeign + Retry	Gateway → 모든 서비스	즉각 응답 필요
동기 REST	OpenFeign	Chat → AI Service	RAG 응답 즉시 필요
동기 REST	OpenFeign	Chat → Todo Service	할일 생성 즉시 확인
비동기 Kafka	Event Publish	Webhook → Notification	수신 즉시 저장·알림
비동기 Kafka	Event Subscribe	Notification → AI Service	임베딩 비동기 처리
비동기 Kafka	Event Subscribe	Notification → Push Service	푸시 발송 분리
비동기 Kafka	Event Subscribe	Notification → Analytics	통계 비동기 집계

3. 서비스별 상세 설계

3.1 API Gateway (:8080)

API Gateway :8080

모든 외부 트래픽의 단일 진입점. 인증·라우팅·보안 담당.

Tech Stack

- Spring Cloud Gateway 4.x
- Spring Security + JWT
- Resilience4j (Circuit Breaker)
- Micrometer + Prometheus

기능

- JWT 검증 및 X-User-Id 헤더 전파
- 서비스별 라우팅 규칙 관리
- Rate Limiting (Redis 기반)
- Circuit Breaker / Fallback 응답

- Request/Response 로깅

라우팅 경로	대상 서비스	인증	비고
/api/v1/notifications/**	Notification :8081	JWT 필수	
/api/v1/webhook/**	Webhook :8082	없음	서명 검증은 서비스 내
/api/v1/chat/**	Chat :8083	JWT 필수	SSE 스트리밍 지원
/api/v1/todos/**	Todo :8084	JWT 필수	
/api/v1/analytics/**	Analytics :8085	JWT 필수	
/api/v1/auth/**	Auth :8086	없음	로그인·토큰 발급
/api/v1/push/**	Push :8087	JWT 필수	내부 전용

3.2 Notification Service (:8081)

Notification Service

알림 원본 저장·조회·상태 관리. Kafka 이벤트 발행.

Tech Stack

- Spring Boot 3.x / Java 21
- Spring Data JPA + QueryDSL
- Spring Kafka (Producer)
- PostgreSQL — notification_db (pgvector)
- Redis (미읽음 카운트 캐시)

기능

- 알림 저장 (webhook.received 소비)
- 목록 조회 (소스 필터 · 페이지네이션)
- 읽음 처리 (단건 / 전체)
- 소프트 삭제 (deleted_at)
- Kafka: notification.created 발행

DB 분리 원칙: Notification Service는 자체 notification_db를 소유한다. 다른 서비스는 이 DB에 직접 접근하지 않으며 REST API 또는 Kafka 이벤트로만 접근한다. pgvector 확장이 필요한 유일한 DB이며, AI Service는 읽기 전용으로만 접근한다.

3.3 Webhook Service (:8082)

Webhook Service

외부 소스 Webhook 수신·검증·파싱. Notification Service로 이벤트 전달.

Tech Stack

- Spring Boot 3.x / Java 21
- Spring Kafka (Producer)
- Spring Security (서명 검증 — 전략 패턴)
- PostgreSQL — webhook_db

기능

- 소스별 Webhook 수신 (/webhook/{source})
- HMAC-SHA256 / Bearer 서명 검증 (WebhookVerifier)
- 페이로드 파싱 → NotificationEvent 변환
- Kafka: webhook.received 발행
- 소스 연결 상태 관리

소스	인증 방식	**Kafka Topic**	파싱 결과
Claude Code	Bearer Token	webhook.received	source=CLAUDE, 작업완료/실패

소스	인증 방식	Kafka Topic	파싱 결과
Slack	HMAC-SHA256	webhook.received	source=SLACK, channel/user/text
GitHub	HMAC-SHA256	webhook.received	source=GITHUB, event/repo
Gmail	Google OIDC JWT	webhook.received	source=GMAIL, subject/from
내부 앱	Bearer Token	webhook.received	source=INTERNAL, 커스텀 페이로드

3.4 AI Service — Python FastAPI (:8090)

Python 분리 이유: LangChain · Hugging Face · 파인튜닝 등 AI/ML 생태계는 Python이 압도적이다. AI Service만 Python FastAPI로 분리하여 Java 서비스들이 REST로 호출한다. Java의 안정성과 Python AI 생태계를 동시에 활용하는 하이브리드 전략.

AI Service :8090

RAG 파이프라인 · LLM 추론 · 임베딩 생성 · 요약 처리 전담.

Tech Stack

- Python 3.12 / FastAPI
- LangChain (RAG 파이프라인)
- Ollama (llama3.2:3b + nomic-embed-text)
- pgvector (벡터 검색 — 읽기 전용)
- Redis (요약 캐시)
- Celery (비동기 임베딩 처리)

기능

- POST /embed — 텍스트 임베딩 생성 (768d)
- POST /retrieve — top-k 유사 알림 검색
- POST /chat — RAG 기반 응답 생성
- GET /stream — SSE 스트리밍 응답
- POST /summarize — 알림 배치 요약
- POST /classify — 우선순위 분류
- GET /report/weekly — 주간 분석 리포트

엔드포인트	호출 서비스	처리 방식	모델
POST /embed	Notification (Kafka→Celery)	비동기	nomic-embed-text (768d)
POST /retrieve	Chat Service	동기	pgvector cosine 검색
POST /chat	Chat Service	동기	llama3.2:3b
GET /stream	Chat Service	SSE	llama3.2:3b
POST /summarize	Chat Service (daily)	동기	llama3.2:3b
POST /classify	Notification Service	Kafka	llama3.2:3b
GET /report/weekly	Analytics Service	동기	llama3.2:3b

3.5 Chat Service (:8083)

Chat Service

채팅 세션 관리 · AI Service 연동 · 할일 생성 컨텍스트 처리.

Tech Stack

- Spring Boot 3.x / Java 21
- Spring WebFlux (SSE 프록시)
- OpenFeign → AI Service

기능

- 채팅 요청 → AI Service 위임
- SSE 스트리밍 Flutter로 프록시
- 오늘 요약 (Redis 캐시 24h)

- OpenFeign → Todo Service
- PostgreSQL — chat_db
- Redis (세션 캐시, 일일 요약 24h)
- PendingTodoContext 컨텍스트 처리
- 채팅 이력 저장·조회 (세션 기반)

3.6 Todo Service (:8084)

Todo Service

알림 기반 할일 생성·관리. 내부 앱 연동 확장 지점.

Tech Stack

- Spring Boot 3.x / Java 21
- Spring Data JPA
- Spring Kafka (Consumer)
- PostgreSQL — todo_db

기능

- POST /todos — 할일 생성 (알림 기반)
- GET /todos — 목록 조회 (상태 필터)
- PATCH /todos/{id} — 상태 변경
- Kafka: todo.created 발행
- 내부 앱 hook 연동

3.7 Push Service (:8087)

Push Service

FCM/APNs 푸시 발송 전담. Kafka 이벤트 소비.

Tech Stack

- Spring Boot 3.x / Java 21
- Firebase Admin SDK (FCM)
- APNs HTTP/2 클라이언트
- Spring Kafka (Consumer)
- PostgreSQL — push_db

기능

- Kafka notification.created 소비 → 푸시 발송
- POST /devices/register — FCM 토큰 등록
- FCM Android 푸시 발송
- APNs iOS 푸시 발송
- 발송 실패 재시도 (Exponential Backoff)
- 토큰 만료 자동 처리

3.8 Analytics Service (:8085)

Analytics Service

알림 패턴 집계 및 LLM 기반 주간 리포트 생성.

Tech Stack

- Spring Boot 3.x / Java 21
- Spring Kafka (Consumer)
- OpenFeign → AI Service
- PostgreSQL — analytics_db
- Redis (집계 캐시)

기능

- Kafka notification.created 소비 → 집계
- GET /analytics/weekly — 주간 통계
- 소스별 / 시간대별 분포 집계
- AI Service → LLM 주간 리포트
- @Scheduled 매주 월요일 자동 생성

4. 인프라 설계

4.1 DB 분리 전략 (Database per Service)

> Database per Service 패턴: 각 서비스는 자체 DB 인스턴스를 소유한다. 공유 DB는 서비스 간 결합도를 높여 독립 배포를 방해하는 MSA 안티패턴이다. 타 서비스 데이터가 필요하면 REST API 또는 Kafka 이벤트로 접근한다.

서비스	DB 인스턴스	주요 테이블	특이사항
Notification	notification_db (PostgreSQL)	notifications, notification_embeddings	pgvector 확장 필요
Webhook	webhook_db (PostgreSQL)	webhook_sources	
Chat	chat_db (PostgreSQL)	chat_messages	
Todo	todo_db (PostgreSQL)	todos	
Push	push_db (PostgreSQL)	devices	FCM 토큰 관리
Analytics	analytics_db (PostgreSQL)	analytics_events	시계열 집계
Auth	auth_db (PostgreSQL)	users, refresh_tokens	비밀번호 암호화
AI Service	notification_db (읽기 전용)	notification_embeddings (읽기만)	벡터 검색 예외 허용

4.2 Kafka 토픽 설계

Topic	Producer	Consumer(s)	파티션	보존 기간
webhook.received	Webhook Service	Notification Service	3	7일
notification.created	Notification Service	AI Service · Push Service · Analytics	6	7일
notification.read	Notification Service	Analytics Service	3	3일
embedding.completed	AI Service	Notification Service	3	3일
todo.created	Todo Service	Analytics Service	3	3일
push.failed	Push Service	Notification Service	3	3일

4.3 로컬 개발 환경 — Docker Compose

> docker-compose up 단일 명령으로 전체 스택 실행. 서비스별 독립 포트 할당. Kafka + Zookeeper + 7개 PostgreSQL 인스턴스 + Redis + Ollama 포함.

서비스	이미지	포트	주요 환경변수
api-gateway	notio/gateway:latest	:8080	JWT_SECRET, SPRING_PROFILES=local
notification-service	notio/notification:latest	:8081	NOTIFICATION_DB_URL, KAFKA_BROKERS

서비스	이미지	포트	주요 환경변수
webhook-service	notio/webhook:latest	:8082	SLACK_SIGNING_SECRET, GITHUB_SECRET
chat-service	notio/chat:latest	:8083	AI_SERVICE_URL, REDIS_URL
todo-service	notio/todo:latest	:8084	TODO_DB_URL
analytics-service	notio/analytics:latest	:8085	ANALYTICS_DB_URL, AI_SERVICE_URL
auth-service	notio/auth:latest	:8086	AUTH_DB_URL, JWT_SECRET
push-service	notio/push:latest	:8087	FCM_KEY, APNS_KEY
ai-service	notio/ai:latest	:8090	OLLAMA_URL, PGVECTOR_URL, REDIS_URL
kafka	confluentinc/cp-kafka:7.6	:9092	KAFKA_AUTO_CREATE_TOPICS=true
notification-db	pgvector/pgvector:pg16	:5433	POSTGRES_DB=notification_db
chat-db	postgres:16	:5434	POSTGRES_DB=chat_db
todo-db	postgres:16	:5435	POSTGRES_DB=todo_db
push-db	postgres:16	:5436	POSTGRES_DB=push_db
analytics-db	postgres:16	:5437	POSTGRES_DB=analytics_db
auth-db	postgres:16	:5438	POSTGRES_DB=auth_db
redis	redis:7-alpine	:6379	
ollama	ollama/ollama:latest	:11434	llama3.2:3b, nomic-embed-text

4.4 프로덕션 — Kubernetes

> Oracle Cloud Free Tier (ARM VM 4Core/24GB) 또는 fly.io를 1차 배포 환경으로 사용한다. 각 서비스는 독립 Deployment + Service 오브젝트로 배포된다. HPA로 자동 스케일링.

서비스	Replicas	CPU Req/Limit	Memory Req/Limit	HPA 조건
api-gateway	2	200m / 500m	256Mi / 512Mi	CPU > 70%
notification-service	2	200m / 500m	256Mi / 512Mi	CPU > 70%
webhook-service	2	100m / 300m	128Mi / 256Mi	—
chat-service	2	200m / 500m	256Mi / 512Mi	연결수 > 100
todo-service	1	100m / 300m	128Mi / 256Mi	—
analytics-service	1	100m / 500m	256Mi / 512Mi	—
auth-service	2	200m / 500m	256Mi / 512Mi	CPU > 70%

서비스	Replicas	CPU Req/Limit	Memory Req/Limit	HPA 조건
push-service	2	100m / 300m	128Mi / 256Mi	Kafka Lag > 1000
ai-service	1	500m / 2000m	1Gi / 4Gi	— (Ollama RAM 제약)
kafka	3	500m / 1000m	1Gi / 2Gi	— (StatefulSet)

4.5 서비스 디스커버리 & 설정 관리

항목	도구	설명
서비스 디스커버리	Kubernetes Service / Ingress	클러스터 내부 DNS. 외부는 Ingress Controller
설정 관리	Spring Cloud Config Server	Git 저장소 기반 서비스별 설정 중앙화
시크릿 관리	Kubernetes Secrets + Sealed Secrets	DB 비밀번호 · API 키 · JWT 시크릿 암호화
헬스체크	Spring Actuator /health	Kubernetes Liveness / Readiness Probe
로드밸런싱	API Gateway (Spring Cloud LB)	Round-Robin. 서비스별 가중치 설정 가능
Circuit Breaker	Resilience4j (Gateway 내장)	50% 실패율 기준. Fallback 응답 반환

5. 관측성 (Observability)

5.1 모니터링 스택

항목	도구	수집 방법	용도
메트릭	Prometheus + Grafana	Spring Actuator /metrics	CPU · 메모리 · 요청수 · 레이턴시
로그	Loki + Grafana	Logback JSON → Promtail	서비스별 로그 중앙화
분산 트레이싱	Zipkin (Micrometer)	HTTP B3 헤더 자동 전파	서비스 간 요청 전 구간 추적
장애 알림	Alertmanager	Prometheus 규칙 기반	Slack 장애 알림 연동
Kafka 모니터링	Kafka UI	Kafka JMX 메트릭	토픽 Lag · 처리량 확인

5.2 분산 트레이싱

> MSA에서 단일 요청이 여러 서비스를 거치므로 Trace ID로 전 구간을 추적한다. Spring Boot 3.x는 Micrometer Tracing이 HTTP traceparent 헤더를 자동 전파한다. AI Service(Python)는 opentelemetry-sdk로 동일 Trace ID를 이어받는다.

구간	Trace 전파	수집 대상
Flutter → API Gateway	HTTP traceparent 헤더	Gateway 수신 레이턴시
Gateway → 각 서비스	HTTP traceparent 헤더 (자동)	라우팅·인증 처리 시간
Chat → AI Service	HTTP traceparent 헤더	RAG 검색·LLM 추론 시간
서비스 → Kafka	Kafka Header (b3) 전파	이벤트 발행·소비 지연

6. CI/CD 파이프라인

> MSA CI/CD 핵심: 서비스별 독립 파이프라인. Notification Service 코드 변경이 Chat Service 배포를 트리거하지 않는다. 각 서비스는 자체 Dockerfile과 GitHub Actions 워크플로우를 보유한다.

6.1 파이프라인 단계

단계	도구	트리거	범위
코드 품질	Checkstyle · SpotBugs (Java) / pylint · mypy (Python)	PR 생성	각 서비스 독립
단위 테스트	JUnit 5 + Mockito (Java) / pytest (Python)	PR 생성·push	각 서비스 독립
통합 테스트	Testcontainers (Java/Python)	main 머지	각 서비스 독립
빌드	Gradle (Java) / Poetry (Python) + Docker buildx	main 머지	각 서비스 독립
이미지 Push	GitHub Container Registry (ghcr.io)	main 머지	각 서비스 독립
dev 배포	kubectl apply / Helm Chart	main 머지	자동 배포
prod 배포	GitHub Actions (수동 승인)	수동 트리거	전체 또는 선택

6.2 배포 전략

환경	전략	설명	롤백
개발 (dev)	Rolling Update	main 머지 시 자동 배포. 다운타임 없음.	kubectl rollout undo
프로덕션 (prod)	Blue-Green	구버전(Blue) 유지 중 신버전(Green) 배포 후 전환.	LB를 Blue로 즉시 전환
AI Service	Canary (10%→100%)	신 모델을 10% 트래픽으로 검증 후 전체 전환.	10% → 0% 즉시 롤백

7. 기술스택 종합 정리

분류	서비스	언어	프레임워크	주요 라이브러리
Gateway	API Gateway	Java 21	Spring Cloud Gateway 4.x	Resilience4j, JWT, Micrometer
Core	Notification Service	Java 21	Spring Boot 3.x	JPA, QueryDSL, Kafka, pgvector
Core	Webhook Service	Java 21	Spring Boot 3.x	Kafka, HMAC 검증, Security
Core	Chat Service	Java 21	Spring Boot 3.x + WebFlux	OpenFeign, Redis, Kafka
Core	Todo Service	Java 21	Spring Boot 3.x	JPA, Kafka
Platform	Auth Service	Java 21	Spring Boot 3.x	Spring Security, JWT, BCrypt
Platform	Push Service	Java 21	Spring Boot 3.x	Firebase Admin, APNs, Kafka
Platform	Analytics Service	Java 21	Spring Boot 3.x	Kafka, OpenFeign, Scheduler
AI	AI Service	Python 3.12	FastAPI	LangChain, Ollama, pgvector, Celery
Client	Flutter App	Dart 3.x	Flutter	Riverpod, go_router, Dio, Drift, FCM
Infra	Message Broker	—	Apache Kafka 3.6	Zookeeper / KRaft
Infra	Primary DB	—	PostgreSQL 16	pgvector (Notification DB)
Infra	Cache / Queue	—	Redis 7	Pub/Sub, Sorted Set, List
Infra	LLM Runtime	—	Ollama	llama3.2:3b, nomic-embed-text
Infra	Container Orch.	—	Kubernetes	Helm, Ingress-NGINX, HPA

분류	서비스	언어	프레임워크	주요 라이브러리
Obs.	Metrics	—	Prometheus + Grafana	Spring Actuator, PromQL
Obs.	Logging	—	Loki + Promtail	Logback JSON
Obs.	Tracing	—	Zipkin	Micrometer Brave, OpenTelemetry
CI/CD	Pipeline	—	GitHub Actions	Docker buildx, ghcr.io, kubectl
Config	Config 관리	—	Spring Cloud Config	Git 저장소 기반 중앙화

8. 향후 확장 시나리오

8.1 멀티 유저 / SaaS 전환

Auth Service + 사용자별 데이터 격리로 SaaS 전환 가능. Notification Service에 user_id 컬럼 추가. API Gateway에서 JWT의 userId를 X-User-Id 헤더로 하위 서비스에 전파.

8.2 새 알림 소스 추가

Webhook Service에 WebhookHandler 구현체만 추가. 기존 코드 무수정(OCP). 예정: JiraWebhookHandler, LinearWebhookHandler, DiscordWebhookHandler.

8.3 LLM 모델 교체

AI Service의 LlmProvider 인터페이스 구현체만 교체. 환경변수 AI_MODEL_PROVIDER=ollama|claude|openai 로 활성화. Fallback 체인: Ollama → Claude API → OpenAI. Canary 배포로 모델 A/B 테스트 가능.

8.4 실시간 협업 — WebSocket Service

WebSocket Service를 신규 마이크로서비스로 추가. Spring WebSocket + STOMP. Kafka 이벤트를 구독하여 연결된 클라이언트에 실시간 푸시. Flutter 앱 탭 추가.

8.5 확장 우선순위

순위	확장 항목	트리거 조건	신규 서비스
1	AI Service Python 분리	RAG 고도화 또는 파인튜닝 필요 시	ai-service (FastAPI)
2	Auth Service 추가	멀티 유저 기능 필요 시	auth-service
3	Webhook / Notification 분리	소스 5개 이상, 독립 배포 필요 시	webhook-service
4	Analytics 분리	통계 쿼리가 메인 DB에 부하 줄 때	analytics-service
5	WebSocket Service	실시간 알림 기능 요구 시	websocket-service
6	Fine-tuning Service	개인화 모델 학습 필요 시	finetuning-service (GPU)

Notio MSA 아키텍처 설계서 v1.0 | 본 설계는 모놀리스 → MSA 점진적 전환(Strangler Fig)을 전제로 한다.